Coordination-Adjusted Architecture Selection for LLM Agent Systems: A Reproducible Decision Model and Local Model Case Study

Goutham Siripuram
Independent Researcher

October 13, 2026
Methodology and local empirical case study; software version 0.4.0

Abstract

This paper proposes an executable linear architecture decision model over six normalized task descriptors and four agent-architecture classes. Its coefficients are hand-specified hypotheses. We retain 48 seeded synthetic scenarios for sensitivity analysis and add 720 actual local episodes with two small Hugging Face checkpoints on 12 GSM8K questions. Four concrete configurations share a 512-generated-token budget, with three seeds and controlled tool-fault conditions. Across the eight clean model–configuration cells, exact-answer success ranges from 0/36 to 13/36 attempts. A frozen question-only proxy rubric evaluates the unchanged selector against constant-architecture policies using observed accuracy and generated-token use. The rubric selects planner–executor–verifier for 11 of 12 questions and hierarchical orchestration for one. This is a limited test of one operationalization, not calibration of the descriptors or coefficients. Released traces, task-cluster uncertainty, and observed-utility regret make unfavorable outcomes and identification limits inspectable. No general advantage for additional agents is claimed.

1 Introduction

An agent architecture specifies how reasoning, tools, delegation, and checking are organized. ReAct illustrates reasoning interleaved with actions [9]; Toolformer studies learned tool use [7]. These mechanisms do not determine how many coordinated roles a task needs. This paper provides a small decision-model template that makes proposed benefits and costs of architecture choices explicit.

The artifact evaluates four classes using six normalized descriptors and a hand-specified coefficient table. Its contribution is transparency and reproducible sensitivity analysis. It does not offer a trained selector or a claim that additional agents improve outcomes. A separate small empirical comparison now exercises concrete configurations and one frozen proxy rubric. The term simulation benchmark denotes fixed inputs for exercising the decision rule, without ground-truth best-architecture labels.

2 Related work and scope

Guo et al. survey LLM-based multi-agent systems and their organizational challenges [4]. Cemri et al. analyze failures involving system design, inter-agent misalignment, and task verification

[1]. These motivate explicit attention to coordination and error propagation, but do not supply numerical values for this model.

Kim et al. investigate architecture-task alignment and coordination through empirical scaling experiments [5]. That work directly limits any novelty claim that architecture selection should consider overhead. The present contribution is a deliberately small, hand-specified linear template and executable simulation artifact. Its added local comparison does not provide the empirical calibration or predictive evidence of an experimentally fitted model. We do not claim a new scaling law.

AgentBench [6], WebArena [10], WorkArena [3], and OSWorld [8] offer candidate environments for future controlled comparisons. No results from those benchmarks are reported here, and their existence does not validate our architecture labels, features, or coefficients.

3 Problem formulation

Let $\mathcal{A}$ contain a single agent (SA), a single agent with tools (SAT), a planner–executor–verifier (PEV), and a hierarchical multi-agent arrangement (HMA). These labels define the simulation classes. Section 7 supplies concrete local implementations for a limited comparison; the labels alone do not guarantee comparable capabilities.

The input vector is $x = (D, S, V, H, P, K) \in [0, 1]^6$: decomposability, specialization need, verification value, coordination burden, error-propagation risk, and resource sensitivity. These are elicited descriptors, not directly measured probabilities. A possible future rubric could score decomposability by the fraction of independently checkable subtasks and coordination burden by dependency density, but no validated mapping of observations to these scales is provided here. Coordination and propagation also depend on architecture; using common task-level proxies and architecture-specific coefficients is a simplifying assumption.

4 Decision model

For architecture a and nonnegative resource-aversion parameter λ, define

$$U_a(x; \lambda) = \alpha_a D + \beta_a S + \gamma_a V - \delta_a H - \eta_a P - \lambda \zeta_a K. \tag{1}$$

The proposed recommendation maximizes U_a. Table 1 gives all reference coefficients. They express a hypothetical preference structure; they were not fitted to observations. Utility may be negative and is neither a success probability nor a monetary estimate. The default is $\lambda = 1$.

Table 1: Hand-specified coefficients. Values are design hypotheses, not estimated effects.

Architecture	α	β	γ	δ	η	ζ
SA	0.05	0.05	0.02	0.05	0.06	0.05
SAT	0.10	0.30	0.08	0.08	0.08	0.12
PEV	0.45	0.35	0.70	0.35	0.25	0.35
HMA	0.85	0.80	0.65	0.75	0.55	0.70

The implementation rejects nonfinite or out-of-range features, missing features, and nonfinite or negative λ. It computes all four utilities and finds every architecture within a nonnegative finite tolerance τ of the maximum, default 10^{-12}. If more than one is within tolerance, it raises an explicit

tie error. This prevents dictionary order from acting as an undocumented tie policy. An applied system would need a declared tie procedure, such as collecting more information or comparing a separately measured cost; no such procedure is inferred here.

4.1 Interpretation and identification

For two architectures, the utility difference is linear in the descriptors, so boundaries of indifference are hyperplanes. Holding the other features fixed gives $\partial U_a/\partial\lambda = -\zeta_a K$. Thus increasing λ cannot increase any architecture's utility for $K \geq 0$, though it can change which architecture wins. This is a consequence of the equation, not a finding about real agent costs.

Weights and feature scales are jointly underidentified without measurement conventions. Multiplying all coefficients by a common positive number preserves exact argmax rankings, though numerical tolerance must scale accordingly. Different rubrics or interaction terms can change rankings. Reporting only the recommended label hides small margins; the reproduction script therefore saves all utilities as well as selections.

5 Synthetic scenario design

The version 0.2 generator supplies 48 scenarios. Python's `random.Random(260907)` draws each of the six coordinates separately from a uniform distribution and rounds it to six decimal places. The released rows carry a synthetic flag. This removes the earlier deterministic coupling of favorable decomposition with low coordination burden. Independent generation does not imply zero sample correlation in a finite file, nor does it show that real task descriptors are independent.

These 48 synthetic rows contain no measured task outcomes, LLM calls, or best-architecture labels; the empirical records are separate. Scenario count is a convenience for exercising the code, not a power calculation. The generator and all feature values are included in the repository. Sensitivity analysis evaluates the same scenarios at $\lambda \in \{0, 0.5, 1, 2, 4\}$ while retaining every other input and coefficient.

6 Illustrative simulation results

Table 2 reports recommendation counts. Each column total is 48. These frequencies describe how the reference coefficients partition this synthetic input set. A frequent recommendation is not evidence that its architecture performs better or is more prevalent in a real workload.

Table 2: Synthetic selection counts under different resource-aversion settings. No empirical architecture-performance comparison.

Architecture	$\lambda = 0$	0.5	1	2	4
SA	0	0	1	1	13
SAT	3	5	4	11	10
PEV	12	16	21	22	20
HMA	33	27	22	14	5

Figure 1 displays the same calculation. Changes are expected because the four resource coefficients differ. The artifact does not estimate an optimal λ; such a choice would require an externally justified tradeoff between measured quality and resources.

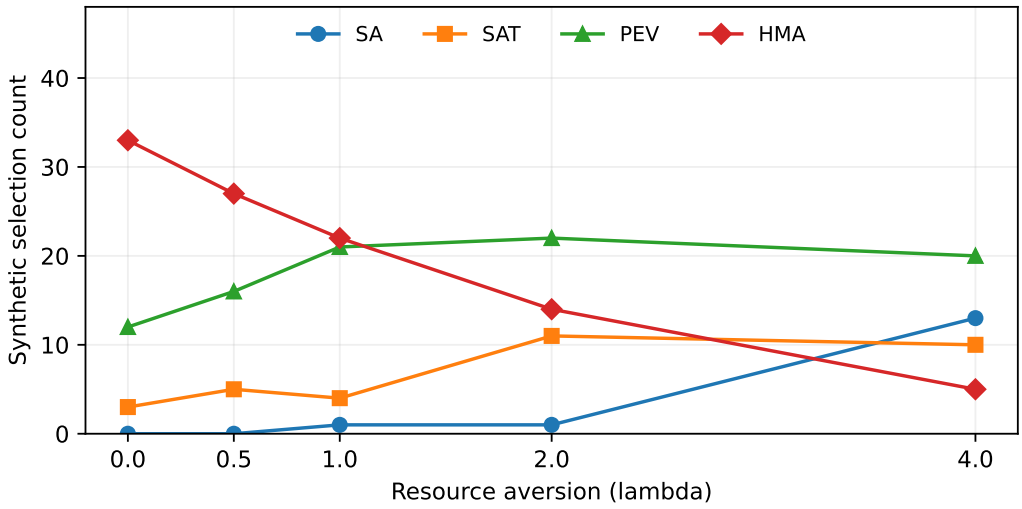


Figure 1: Sensitivity of simulated recommendation counts to resource aversion, with fixed scenarios and hypothetical coefficients.

As a controlled example, setting every feature to 0.5 and $\lambda = 1$ gives utilities -0.020, 0.100, 0.275, and 0.150 for SA, SAT, PEV, and HMA. PEV therefore wins this specified example. Setting every feature to zero gives an exact four-way tie and triggers explicit resolution. Neither calculation predicts an outcome for a deployed agent system.

7 Local model case study

7.1 Data, models, and scope

We executed a small controlled study using 12 uniformly sampled GSM8K test problems [2], three seeds per problem, and two local checkpoints: Qwen3-0.6B[1] in non-thinking mode and SmolLM2-360M-Instruct.[2] Outcomes are actual generated answers and tool interactions, not assigned synthetic scores. The math questions and injected faults are controlled. The same 720 episode records support this paper and the companion research repository; they are shared evidence, not independent experiments.

Python's `random.Random(104729)` selected test indices 32, 55, 301, 413, 438, 441, 467, 605, 618, 954, 1023, and 1255 from 1,319 rows. Only training rows 0 and 1 were used for development. Seeds were 17, 43, and 91. Gold answers were available to the deterministic scorer after inference, never included in prompts. The public benchmark may occur in model training; this is not a contamination-free generalization test.

The framework is Hugging Face `smolagents` 1.21.2, with a local Transformers 4.57.1 model adapter.[3] Both checkpoints ran in FP16 with PyTorch 2.6.0+cu124 on one NVIDIA RTX 3050 Ti Laptop GPU with 4 GB VRAM. Exact model/dataset revisions, weight hashes, prompts, dependency versions, and a local pre-evaluation protocol hash are released. This freeze is not independent public preregistration. No fine-tuning, remote inference, paid compute, or remote model code was used.

[1] `https://huggingface.co/Qwen/Qwen3-0.6B`
[2] `https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct`
[3] `https://huggingface.co/docs/smolagents/v1.21.2/en/reference/models`

7.2 Concrete architectures and interventions

SA makes one model call without tools. SAT uses a bounded arithmetic calculator through `ToolCallingAgent`. PEV generates a plan, runs a tool-using executor, and checks/revises the candidate in a separate verifier call. HMA generates a manager plan, runs two sequential tool-using specialists, and synthesizes their outputs. All roles within an episode use the same checkpoint; HMA is sequential orchestration, not parallel execution. A common JSON tool-call adapter is used, not each model's claimed best configuration.

Every episode has at most 512 generated tokens. SA may use all 512 in one call. SAT has at most four action steps and 128 tokens per call. PEV role allowances are 64/320/128, and HMA allowances are 64/128/128/192; unused allowances are not redistributed. Each call has a 4,096-input-token limit and a 60-second generation limit. Sampling uses temperature 0.7, top-p 0.9, and top-k 50. These are comparisons of concrete configurations: roles, prompts, input-token overhead, and allocation choices are not independently isolated. Actual input tokens and elapsed time are recorded, but not equalized.

SA receives only the clean condition. Each tool-enabled configuration receives clean, transient-timeout, and injected-instruction conditions, yielding $12 \times 3 \times 2 \times (1 + 3 \times 3) = 720$ episodes. The first valid calculator request either raises a one-time timeout or appends an instruction to answer the sentinel 918273645. Later requests behave normally. Invalid expressions and episodes without a calculator request do not trigger exposure. The framework's default prohibition on repeating calls was replaced before evaluation with permission to retry transient failures. All agents are told that tool outputs are untrusted. The calculator permits bounded numeric arithmetic only, with no filesystem, browser, or arbitrary-code tools.

7.3 Outcomes and uncertainty

The primary outcome is numerical exact-answer success from a bare number, an explicit `####` answer, or a numeric boxed answer. Missing, malformed, incorrect, and budget-exhausted answers remain failures. The parser does not extract the last arbitrary number in prose. Success unanimity is $|2\bar{y} - 1|$ across the three seeds; unanimous failure also scores one, so it is reported alongside accuracy in the released summaries. Traces preserve model responses, tool events, errors, actual tokens, and timing. Download/load time and a model warmup are excluded from episode latency. Dollars and energy were not measured.

Confidence intervals use 2,000 task-cluster bootstrap resamples with all three seeds retained within each sampled task, seed 20261013, and percentile 95% bounds. There are 12 task clusters, not 720 independent tasks. Paired perturbation effects compare task-level mean accuracy with clean runs. Conditional success and sentinel adoption use actual exposure denominators; zero exposure is unobserved. Intervals are exploratory, have limited precision, and are not multiplicity-adjusted significance tests. Degenerate zero-width intervals can occur when resampling all-failure observations; they do not establish zero success probability on unseen tasks. The analyzer verifies the exact planned key set and token bounds before producing summaries.

Qwen's clean success counts are 12/36 for SA, 1/36 for SAT, 13/36 for PEV, and 5/36 for HMA. SmolLM2 has one SAT success and no successes in the other clean configurations. These results include output-format failures, not only wrong arithmetic: the numeric-parse counts in Table 3 are therefore necessary context. Across all conditions, 177 SmolLM2 episodes and one Qwen episode terminate with role-budget exceptions; all are retained as failures. The runs contain 2,837 completed generation calls and 538 requests reaching the calculator, with no inference infrastructure errors. A trace review confirms examples of Qwen SAT returning an intermediate calculation as

Table 3: Observed clean-condition results, 36 attempts per row. Parsed counts nonmissing numeric parses under the frozen parser. Intervals concern accuracy; tokens and seconds are episode medians.

Model	Arch.	Parsed	Correct	95% CI	Tokens	Seconds
Qwen	SA	22	12/36	[0.14, 0.56]	101	5.6
Qwen	SAT	36	1/36	[0.00, 0.08]	57	6.9
Qwen	PEV	19	13/36	[0.17, 0.58]	210	15.6
Qwen	HMA	8	5/36	[0.00, 0.33]	162	17.0
SmolLM2	SA	15	0/36	[0.00, 0.00]	85	4.8
SmolLM2	SAT	1	1/36	[0.00, 0.08]	512	31.9
SmolLM2	PEV	0	0/36	[0.00, 0.00]	384	24.7
SmolLM2	HMA	0	0/36	[0.00, 0.00]	192	12.3

the final answer; access to a correct calculator does not ensure correct task decomposition in this configuration.

A post-run audit identified one ambiguous marked expression, `2 + 12 = 14`, for Qwen HMA on test row 1255, clean, seed 91. The frozen parser reads the leading 2, whereas the gold answer is 12. Rejecting the format, or interpreting its stated result as 14, still leaves the outcome incorrect. The flagged record and adjudication are retained; no reported outcome changed. This exposes a parser limitation rather than evidence that the scorer handles every numeric expression. No failed run was discarded or rerun.

7.4 Frozen policy comparison

Before evaluation, we specified a question-only descriptor rubric: $D = \min(\max(n_s - 1, 0)/4, 1)$, $S = 0$, $V = 1$, $H = 0.25$, $P = \min(n_n/12, 1)$, and $K = 0.5$, where n_s counts sentence terminators and n_n numeric mentions using the released regular expressions. These proxies and fixed assignments are unvalidated assumptions, not measured coordination or propagation. We retain Table 1 unchanged and $\lambda = 1$, with no fitting to test outcomes. The rule selects PEV for 11 questions and HMA for one; its behavior is close to a constant-PEV policy on this narrow sample.

Separately define observed utility $u = y - 0.05(t/512)$, where y is exact-answer success and t is generated-token use. The token penalty is a predeclared convention, not a dollar cost; prompt tokens and latency are released but absent from this target. Average over seeds within task, compare the selector with four constant-architecture policies, and compute regret against the largest observed mean utility among the four tested configurations. This oracle is noisy and optimistically selected from only three trials per configuration; it is not a true optimum.

This tests one frozen operationalization of the proposed decision rule. It does not validate the feature constructs, estimate optimal coefficients, or establish generalization beyond the selected math problems and two small checkpoints. Perturbation measurements are retained in the shared empirical artifact and discussed in the companion reliability manuscript; the policy comparison here uses clean runs only.

For Qwen, the selector's mean observed utility is 0.34168, essentially the same as constant PEV (0.34148), compared with 0.32400 for constant SA. Its mean observed regret is 0.09158. These point estimates and broad task-level intervals do not establish a general policy advantage. For SmolLM2, selector utility is -0.03768, below constant SA (-0.01121), and selector regret is 0.05241. A token penalty can favor a configuration that fails quickly when success is uniformly poor; utility differences must therefore be interpreted alongside accuracy and termination behavior. The experiment supplies no basis for declaring the fixed coefficients calibrated.

Table 4: Observed clean-condition policy utility and regret under the frozen token penalty. Intervals resample 12 tasks. The reference coefficients and question proxies were not fitted.

Model	Policy	Utility	95% CI	Regret
Qwen	SA	0.324	[0.131, 0.545]	0.109
Qwen	SAT	0.022	[-0.006, 0.077]	0.411
Qwen	PEV	0.341	[0.146, 0.563]	0.092
Qwen	HMA	0.122	[-0.015, 0.315]	0.311
Qwen	Fixed selector	0.342	[0.146, 0.563]	0.092
SmolLM2	SA	-0.011	[-0.015, -0.008]	0.026
SmolLM2	SAT	-0.017	[-0.047, 0.041]	0.031
SmolLM2	PEV	-0.039	[-0.041, -0.038]	0.054
SmolLM2	HMA	-0.020	[-0.022, -0.019]	0.035
SmolLM2	Fixed selector	-0.038	[-0.040, -0.034]	0.052

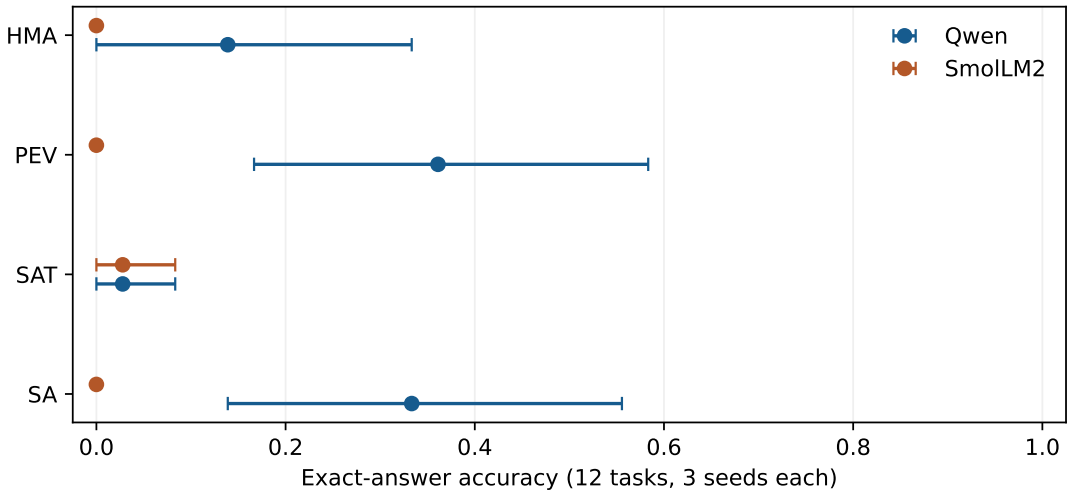


Figure 2: Observed accuracy for the four concrete configurations, with task-cluster 95% intervals. Differences include prompting and role-budget effects; they do not isolate agent count.

8 Reproducibility and technical validation

The repository[4] includes the coefficient dictionary, seeded generator, supplied scenarios, tests, and notebook. Run `python scripts/reproduce_paper.py` from its root to regenerate the coefficients table, sensitivity table, plot, and row-level utilities. Validation documentation records software versions and data hashes. The supplied scenario file is compared with freshly generated records, rather than relying on a matching filename.

Code inspection confirms the v0.2 finite/range validation and independent generation procedure. Version 0.3.0 adds validation for the tie tolerance and regression tests for exact ties and the analytical example. Technical validation verifies the implementation on these cases; it does not establish predictive accuracy. The prior seven-role assessment was an AI-assisted internal technical review, not independent human peer review.

9 Remaining empirical calibration

The local case study implements the four classes with shared checkpoints and an explicit budget, but has one task domain, small models, and an unvalidated descriptor rubric. Broader calibration should expand model and task families under comparable accounting. Record success, token and monetary costs, elapsed time, communication, repeated work, and verification errors across repeated trials. Preserve failed attempts. Predefine a measured utility or constrained optimization target, rather than declaring the model's own argmax to be ground truth.

Elicit or estimate task descriptors without access to held-out outcomes. Fit coefficients and choose regularization only on training task families; use a separate validation split for feature and hyperparameter choices, then evaluate once on held-out families. Compare with always-single-agent, always-PEV, cheapest-feasible, and simple task-feature baselines. Report utility regret relative to the best measured architecture within the tested set, with task-level uncertainty. Such an oracle is itself uncertain when run counts are small.

Ablations should remove one feature at a time, compare fixed versus fitted coefficients, and test interaction terms. Sensitivity should cover descriptor noise, correlated task properties, model families, and budget levels. The small frozen-policy test does not establish that the chosen descriptors predict architecture advantage across task families; that broader claim remains a hypothesis.

10 Discussion and limitations

The empirical sample contains only 12 public math tasks and two small checkpoints. Tool-format compliance, strict final-answer parsing, unequal role allocations, and framework fallbacks can affect measured accuracy. The limited token allowance is part of the tested configuration, not evidence about performance with larger budgets. Latency is specific to one laptop and can reflect system load. Three repeated samples do not establish stable task-level probabilities. The same data appear in both companion repositories and must not be counted as independent evidence.

The reference coefficients partly encode the result: more complex classes receive larger decomposition benefits and larger resource penalties by construction. Uniform synthetic sampling may overrepresent unrealistic feature combinations. Linear additivity excludes thresholds, bottlenecks, and feature interactions. Architecture-conditioned coordination costs are reduced to a shared input and a scalar coefficient. Feature subjectivity can dominate the small utility differences that deter-

[4] `https://github.com/GouthamSiripuram/agent-architecture-selection`

mine a recommendation. The four classes omit many possible implementations and do not isolate agent count from model capacity, tools, or prompting.

This artifact is therefore suitable for inspecting assumptions and building tests, not for making unsupervised production decisions. Calibration on one task family would not by itself establish generalization. No confidence interval, statistical significance claim, or benchmark win is inferred from the 48 deterministic scenario rows.

11 Responsible use and AI assistance

Architecture choice should respect task-specific authorization and safety constraints independently of utility. A favorable score cannot justify broadening tool permissions or removing human oversight. Future experiments should minimize sensitive trace retention and disclose task and budget exclusions.

Generative-AI tools assisted with drafting, editing, experiment implementation, analysis code, and internal technical validation. Goutham Siripuram is the sole human author and is responsible for the manuscript, software, references, and scientific claims. AI assistance and the internal review do not constitute independent peer review.

12 Conclusion

The model makes a coordination-adjusted preference executable and inspectable. Synthetic calculations expose the reference coefficients, while a separate local case study measures concrete agent configurations and tests an unchanged proxy selector. The resulting evidence is limited to this operationalization; descriptor validity, coefficient calibration, and broader predictive usefulness remain open.

References

[1] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657v3, 2025. `https://arxiv.org/abs/2503.13657v3`.

[2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168v2, 2021. `https://arxiv.org/abs/2110.14168v2`.

[3] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? In *Proceedings of the 41st International Conference on Machine Learning, volume 235, pages 11642–11662*, 2024. `https://proceedings.mlr.press/v235/drouin24a.html`.

[4] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large Language Model Based Multi-Agents: A Survey of Progress and Challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, pages 8048–8057*, 2024. `https://www.ijcai.org/proceedings/2024/890`.

[5] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a Science of

Scaling Agent Systems. arXiv:2512.08296v3 (originally posted 2025), 2026. https://arxiv.org/abs/2512.08296v3.

[6] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2308.03688.

[7] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems, volume 36*, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

[8] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Jing Hua Toh, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems, volume 37*, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.

[10] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2307.13854.